CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 10

PHP Login System with Role-Based Access Control, Session Management and Secure Logout

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To build a StudentHub login system in PHP and MySQL with password verification, role-based access control for Admin and Student users, secure session management with an inactivity timeout, and a secure logout.

- Authenticate users by email and password with MySQLi prepared statements and password_verify().
- Regenerate the session ID immediately after a successful login (protection against session fixation).
- Send each role to its own dashboard and refuse the other role's dashboard with “Unauthorized Access”.
- End inactive sessions automatically (timeout.php) and destroy the session completely on logout.
- Make sure that a dashboard cannot be opened again after logout, not even with the Back button.

2. Problem Definition

After registration (Practical 9) users must be able to log in. A careless login system reveals which email addresses are registered, keeps the same session ID before and after login (so an attacker who planted an ID can take over the session), decides access only by hiding links, leaves sessions open forever on shared computers, and lets the browser show a cached dashboard after logout. This practical solves each of these problems on the server.

Starting point: the Practical-10 folder already had the ten required file names. They used the shared user_registration database, printed the active session ID on the dashboards, queried tables (students, events) that do not exist in that database, passed message text through the address, showed placeholder grades and a timetable for the student, and had a remember-me cookie that bypassed the timeout. The files were backed up and rewritten; the file names, password_verify() and session_regenerate_id() were kept.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Sessions, password_verify(), role checks; built-in web server for testing
MySQLi	Database access with prepared statements
MariaDB 10.4.32 (XAMPP), phpMyAdmin 5.2.1	Database studenthub_prac10; export with mysqldump, import with phpMyAdmin
HTML5, CSS3	Login form and dashboards (style.css); no JavaScript is needed
Microsoft Edge	Running the application; headless mode used for the screenshots and the console/Back-button checks
Python 3	tests/test_app.py – automated end-to-end tests (standard library only)

4. Database Design

The practical uses its own database, studenthub_prac10, so the Practical 8 and 9 databases and the main StudentHub database are never changed. The table users stores each account once; the email address is the login name and is therefore UNIQUE. The role column is an ENUM, so only admin or student can be stored,

and it has an index for listing users by role. A CHECK constraint makes MySQL refuse any password_hash value that is not a bcrypt/Argon2 hash.

Column	Type	Notes
user_id	INT UNSIGNED AUTO_INCREMENT	Primary key
name	VARCHAR(100)	Shown on the dashboards
email	VARCHAR(254)	UNIQUE uq_users_email; CHECK email format
password_hash	VARCHAR(255)	password_hash() output; CHECK bcrypt/Argon2
role	ENUM('admin','student')	Default student; index idx_users_role
created_at	DATETIME	Default CURRENT_TIMESTAMP

user_registration.sql is the mysqldump export of this database (CREATE DATABASE, the table and the three fictional accounts: Demo Admin, Demo Student and Demo Student Two, all @example.com). It imports in phpMyAdmin (Import → Choose File → Import). The demo passwords are not in the repository: the hashes in the export come from random passwords that were not kept, and each installation sets its own with tools/set_password.php, which stores only the password_hash() result.

```
45  CREATE TABLE `users` (
46    `user_id` int(10) unsigned NOT NULL AUTO_INCREMENT,
47    `name` varchar(100) NOT NULL,
48    `email` varchar(254) NOT NULL,
49    `password_hash` varchar(255) NOT NULL COMMENT 'password_hash() output, never the password',
50    `role` enum('admin','student') NOT NULL DEFAULT 'student',
51    `created_at` datetime NOT NULL DEFAULT current_timestamp(),
52    PRIMARY KEY (`user_id`),
53    UNIQUE KEY `uq_users_email` (`email`),
54    KEY `idx_users_role` (`role`),
55    CONSTRAINT `chk_users_email` CHECK (`email` like '%_@_%._%'),
56    CONSTRAINT `chk_users_password_hash` CHECK ((`password_hash` like '$2y$%' or `password_hash` like
    '$argon2%') and char_length(`password_hash`) >= 60)
57  ) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

5. Login Implementation

login.php shows a form with an email and a password field and a hidden CSRF token. It is sent to authenticate.php, which accepts only POST requests with a valid token, looks the user up by email with a MySQLi prepared statement and checks the password. Every failure – unknown email, wrong password, empty field – redirects to the same message “Invalid email or password.”, so the page never reveals whether an email is registered. Messages on the login page are chosen from a fixed list by a short code (for example status=invalid); text from the address is never printed. If MySQL cannot be reached, the user sees “Login is not available right now” and the real error goes to the PHP error log.

```
26  try {
27      $stmt = get_db()->prepare('SELECT user_id, name, email, password_hash, role FROM users WHERE email
    = ? LIMIT 1');
28      $stmt->bind_param('s', $email);
29      $stmt->execute();
30      $row = $stmt->get_result()->fetch_assoc();
31      $stmt->close();
32  } catch (RuntimeException $exception) {          // includes mysqli_sql_exception; connection errors
    are logged in db.php
33      if ($exception instanceof mysqli_sql_exception) {
34          error_log('Practical 10 login query failed: [' . $exception->getCode() . '] ' . $exception-
    >getMessage());
35      }
36      redirect('login.php?status=unavailable');
37  }
38
39  // For an unknown email a dummy hash is checked, so both failures take about the same time
40  $hash = $row['password_hash'] ?? '$2y$10$00000000000000000000000.0000000000000000000000000000000';
41  if (!password_verify($password, $hash) || $row === null) {
42      redirect('login.php?status=invalid');
43  }
```

6. Password Verification

Passwords are stored only as password_hash() hashes (bcrypt, $2y$10$…). password_verify($password, $hash) reads the algorithm, cost and salt from the stored hash, hashes the typed password the same way and compares the results; the stored hash cannot be turned back into the password. When the email is unknown, a dummy hash is still checked, so a failed login takes about the same time whether or not the account exists. The tests confirmed that password_verify() returns true for the password set with tools/set_password.php and that the stored value is not the password.

7. Session Management

After the password is verified, session_regenerate_id(true) gives the session a new ID and deletes the old one on the server. An attacker who managed to plant or learn the pre-login ID therefore gains nothing (session fixation). Only then are the user's ID, name, email and role – read from the database, never from the form – stored in $_SESSION, together with the time of the last activity.

```
45  // Successful login: new session ID (the old one is deleted), so an ID known before login is useless
46  $oldId = session_id();
47  session_regenerate_id(true);
48  $_SESSION['user'] = ['id' => (int) $row['user_id'], 'name' => $row['name'], 'email' => $row['email'],
    'role' => $row['role']];
49  $_SESSION['last_activity'] = time();
50  $_SESSION['login_time'] = time();
51  // Evidence for the dashboard: short SHA-256 fingerprints of the old and new ID (not the IDs
    themselves)
52  $_SESSION['id_check'] = ['before' => substr(hash('sha256', $oldId), 0, 12), 'after' =>
    substr(hash('sha256', session_id()), 0, 12)];
53  $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
54
55  redirect(ROLE_DASHBOARDS[$row['role']]);
```

The session cookie P10SESSID is HttpOnly, SameSite=Strict and Secure when HTTPS is used, and session.use_strict_mode refuses IDs that the server did not create. As evidence of the regeneration, dashboard.php shows short SHA-256 fingerprints of the ID before and after login (Figure 9) – the IDs themselves are never displayed. The test script additionally compared the cookie before and after login and confirmed that the old ID no longer gives access.

```php
function start_secure_session(): void
{
    if (session_status() === PHP_SESSION_ACTIVE) {
        return;
    }
    ini_set('session.use_strict_mode', '1');
    ini_set('session.use_only_cookies', '1');
    ini_set('session.use_trans_sid', '0');
    session_name('P10SESSID');
    session_set_cookie_params([
        'lifetime' => 0,
        'path'     => '/',
        'secure'   => !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off',
        'httponly' => true,
        'samesite' => 'Strict',
    ]);
    session_start();
}
```

8. Role-Based Access Control

Every protected page starts with require_login() (dashboard.php) or require_role('admin') / require_role('student'). These functions run on the server before any output: a visitor who is not logged in is redirected to the login page; a logged-in user with the wrong role receives HTTP 403 and the page “Unauthorized Access” instead of the dashboard. Navigation links are only a convenience – entering admin_dashboard.php directly as a Student is refused (Figure 4). A role field added to the login form is ignored, because the role comes from the database.

```php
// Protected pages: not logged in → login page; inactive too long → timeout.php; otherwise the activity
// time is renewed. Returns the logged-in user (id, name, email, role) taken from the session, never from the browser.
function require_login(): array
{
    start_secure_session();
    send_security_headers();
    $user = current_user();
    if ($user === null) {
        redirect('login.php?status=login_required');
    }
    if (session_has_expired()) {
        redirect('timeout.php');
    }
    $_SESSION['last_activity'] = time();
    return $user;
}

// Like require_login(), but a logged-in user with another role receives HTTP 403 "Unauthorized Access"
function require_role(string $role): array
{
    $user = require_login();
    if ($user['role'] !== $role) {
        http_response_code(403);
        page_start('Unauthorized Access', $user);
        echo '<section class="card alert-card" role="alert">', "\n",
             '    <h2 class="denied-title">Unauthorized Access</h2>', "\n",
             '    <p>Your account (role: <strong>', e(ROLES[$user['role']]), '</strong>) is not allowed to open the ',
             e(ROLES[$role]), ' dashboard. The request was refused by the server (HTTP 403).</p>', "\n",
             '    <p><a class="button" href="', e(ROLE_DASHBOARDS[$user['role']]), '">Go to your ',
             e(ROLES[$user['role']]), ' dashboard</a></p>', "\n",
             '</section>', "\n";
        page_end();
        exit;
    }
    return $user;
}
```

9. Session Timeout and Secure Logout

Each protected request stores the current time in $_SESSION['last_activity']. If more than the configured limit has passed since the last request (300 seconds by default; set in config.local.php as session_timeout), require_login() sends the browser to timeout.php. timeout.php destroys the expired session and redirects to the login page with “Your session expired after 5 minutes of inactivity”. Protected pages also contain a meta refresh to timeout.php one second after the limit, so a page left open returns to the login page by itself. timeout.php does not end a session that is still active.

```
 9  start_secure_session();
10  send_security_headers();
11
12  // A session that is still active is not ended here (only logout.php does that)
13  if (current_user() !== null && !session_has_expired()) {
14      redirect('dashboard.php');
15  }
16  end_session();
17  redirect('login.php?status=timeout');
```

logout.php accepts only a POST request with the session's CSRF token (a link on another website cannot log the user out). end_session() empties $_SESSION, sends an expired session cookie with the same settings and calls session_destroy(), so the session no longer exists on the server even if someone kept the old cookie. All pages send Cache-Control: no-store, so the browser must ask the server again for a protected page – after logout the Back button leads to the login page, which was checked in Edge.

```
87  // Clears the session data, expires the session cookie in the browser and deletes the session on the
    server
88  function end_session(): void
89  {
90      $_SESSION = [];
91      $params = session_get_cookie_params();
92      setcookie(session_name(), '', [
93          'expires' => time() - 42000, 'path' => $params['path'], 'domain' => $params['domain'],
94          'secure' => $params['secure'], 'httponly' => $params['httponly'], 'samesite' =>
    $params['samesite'],
95      ]);
96      session_destroy();
97  }
```

```
12  if (($_SERVER['REQUEST_METHOD'] ?? 'GET') !== 'POST' || !csrf_is_valid($_POST['csrf_token'] ?? null)) {
13      redirect(current_user() !== null ? 'dashboard.php' : 'login.php');
14  }
15  end_session();
16  redirect('login.php?status=logged_out');
```

10. Output Screenshots

All screenshots are genuine Microsoft Edge renders of the running application (PHP built-in server, database studenthub_prac10) using the fictional demo accounts. The empty area below the content was cropped for the report; the full images are in Practical-10/screenshots.

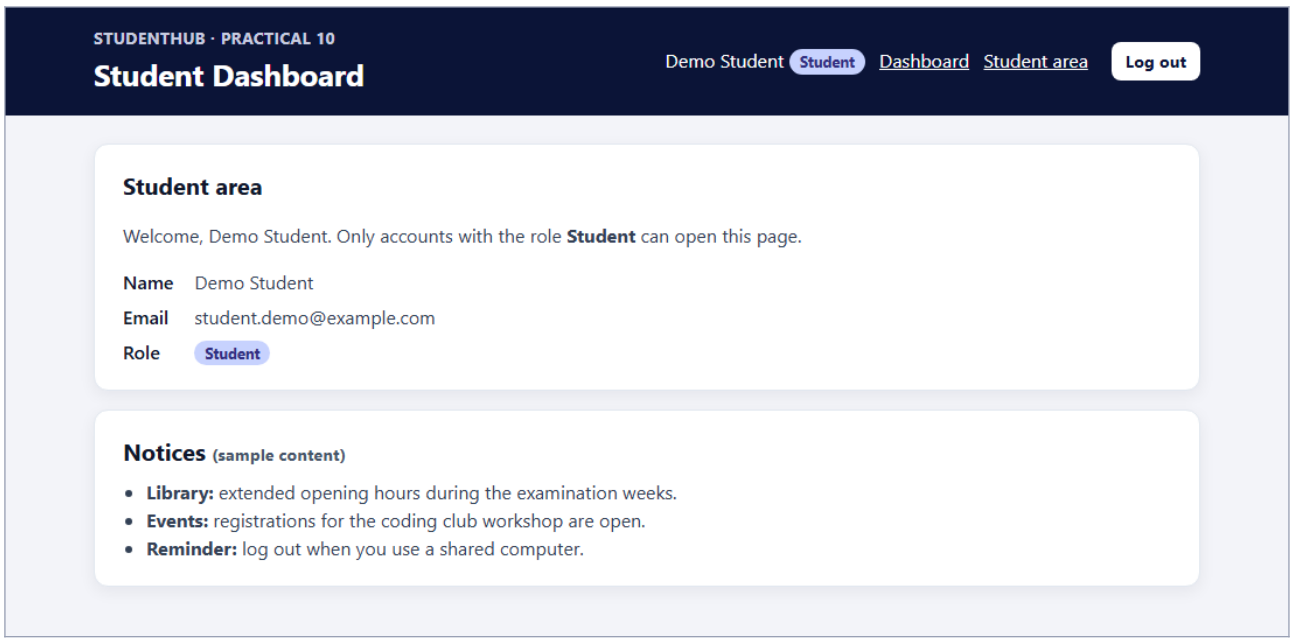


Figure 1: Student login – Demo Student is redirected to student_dashboard.php

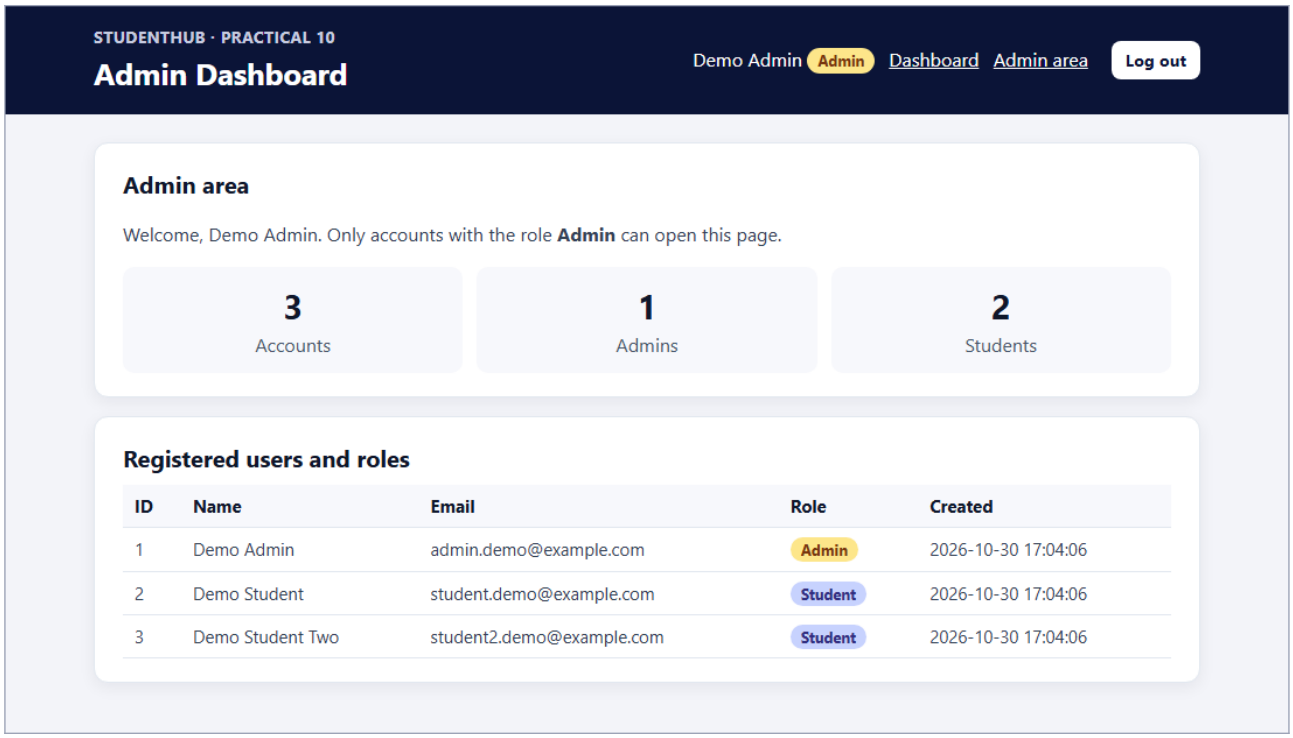


Figure 2: Admin login – Demo Admin is redirected to admin_dashboard.php with the user list and roles

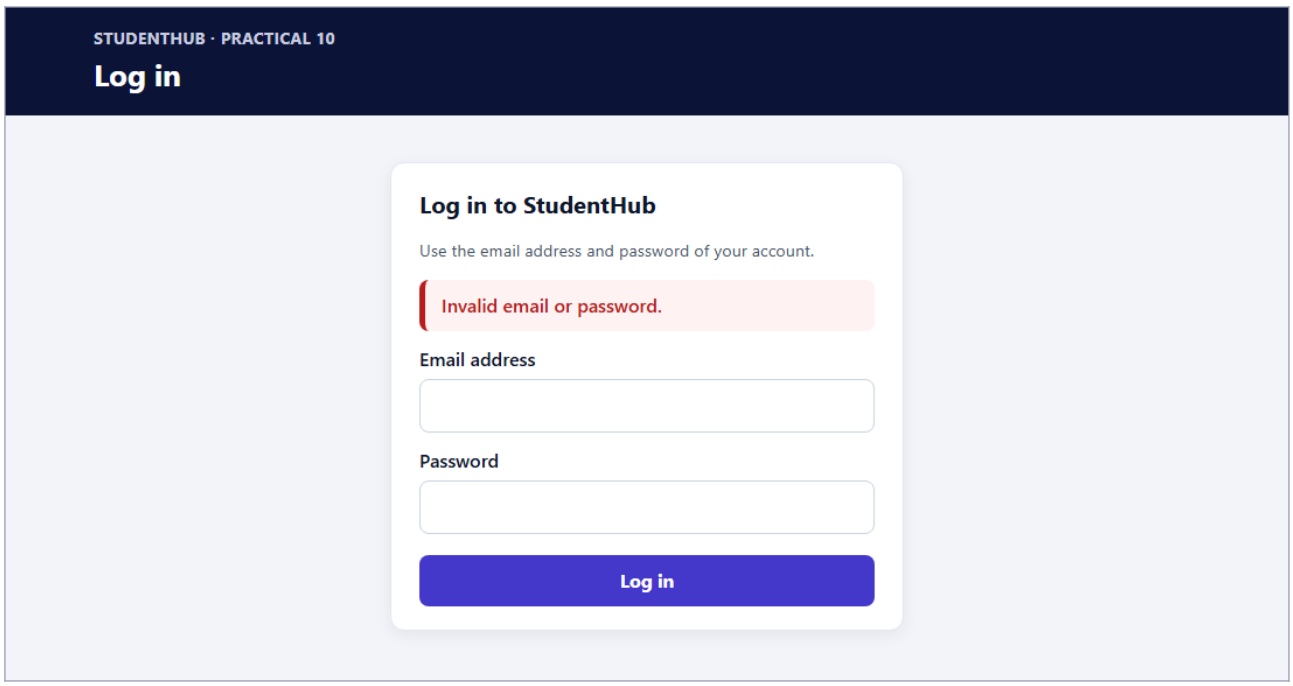


Figure 3: Wrong password – generic message "Invalid email or password."

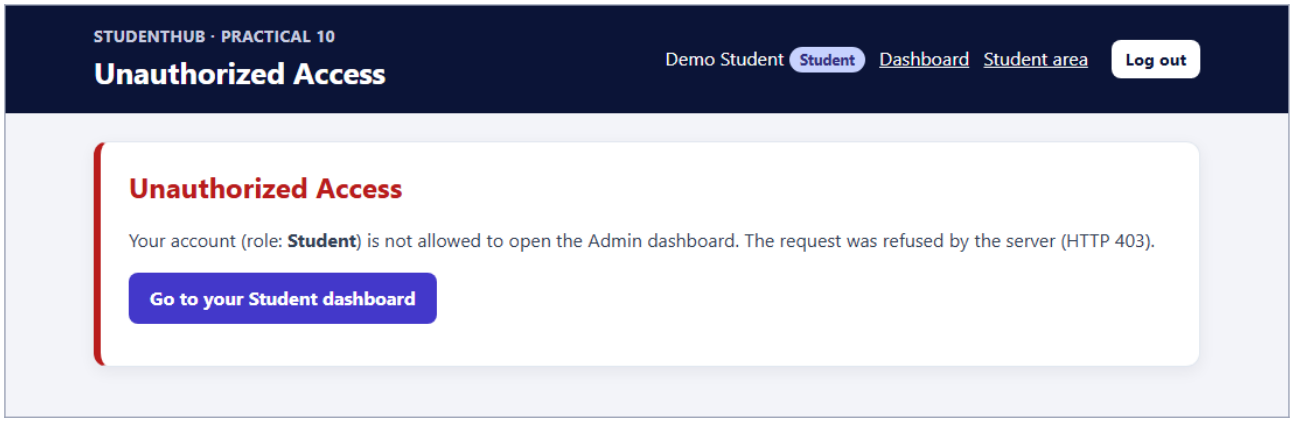


Figure 4: Student opening admin_dashboard.php – HTTP 403 "Unauthorized Access"

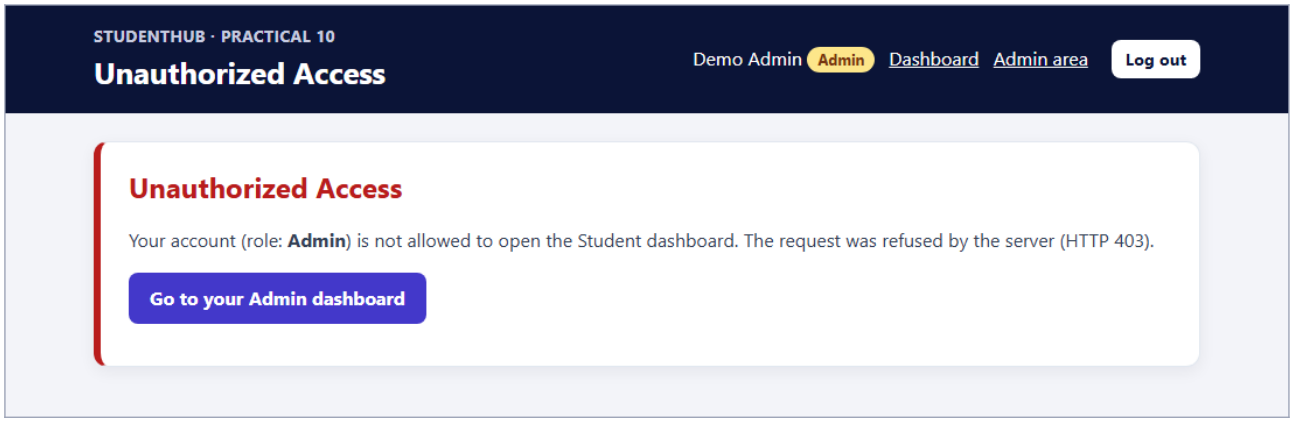


Figure 5: Admin opening student_dashboard.php – HTTP 403 "Unauthorized Access"

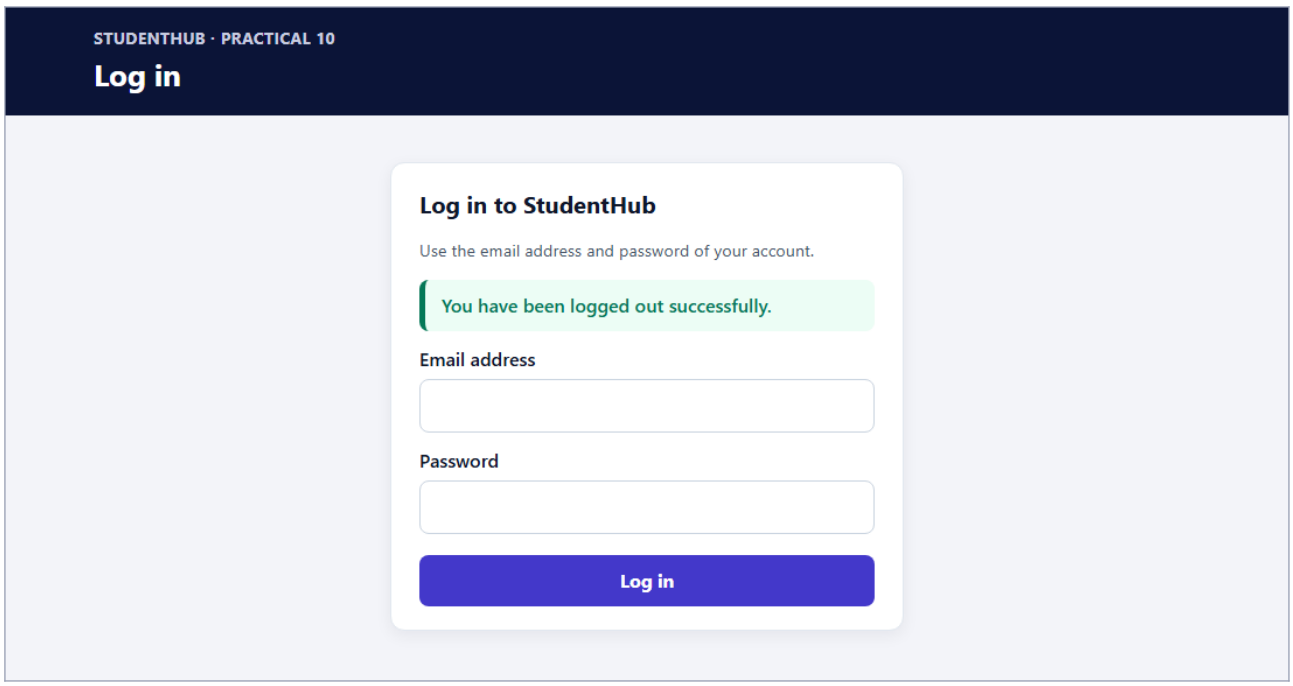


Figure 6: After pressing Log out – "You have been logged out successfully."

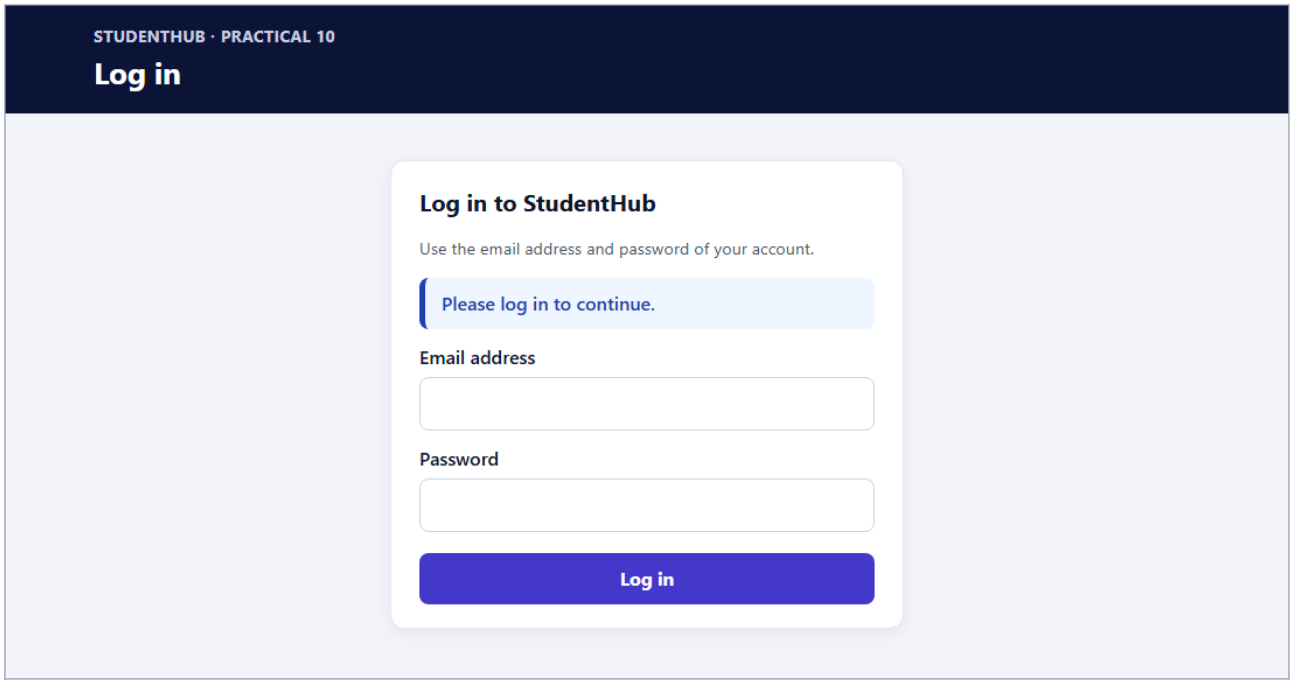


Figure 7: student_dashboard.php opened again after logout – redirected to the login page (the Back button gives the same result)

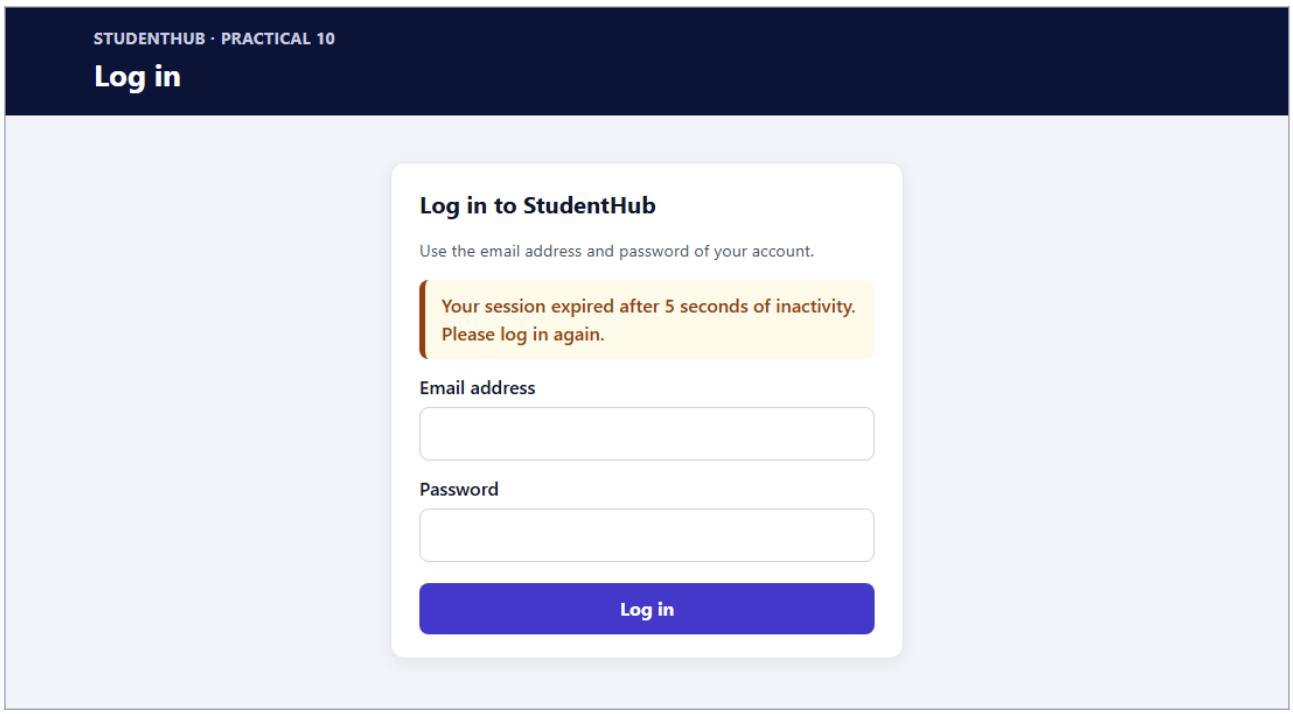


Figure 8: Session timeout – the idle dashboard refreshed itself to timeout.php, which ended the session (server started with a 5-second timeout for this demonstration; the default is 300 seconds)

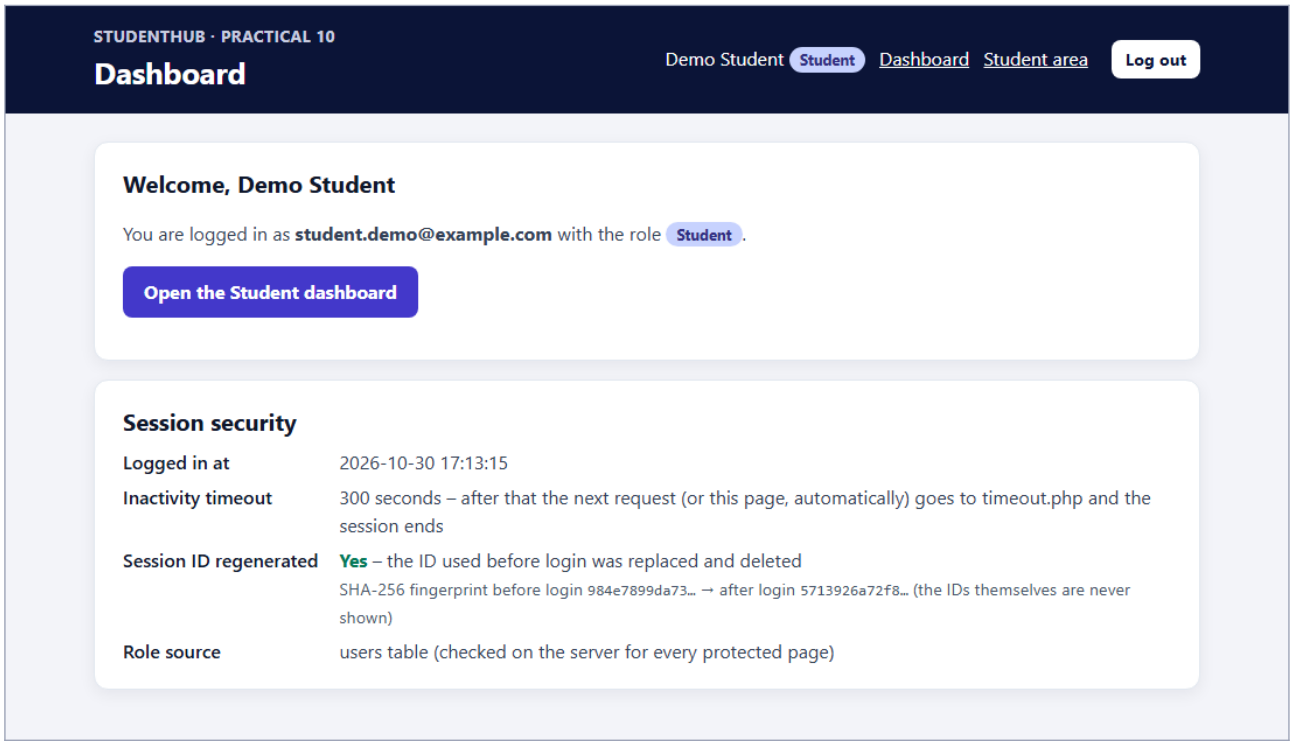


Figure 9: dashboard.php – "Session ID regenerated: Yes" with SHA-256 fingerprints of the old and new ID (the IDs are never shown)

11. Test Cases and Results

tests/test_app.py was executed on the final version (PHP 8.2.12, MariaDB 10.4.32, Windows 11). It creates the disposable database studenthub_prac10_test from user_registration.sql, sets random demo passwords

with tools/set_password.php, runs three PHP servers (normal, 2-second timeout, MySQL unreachable) and removes the test database afterwards; studenthub_prac10 is not changed and no password is printed. Result: 76 of 76 checks passed.

#	Test	Expected	Result
1	php -l on all PHP files; source uses prepare() for every SQL statement	No errors; no query()	Pass
2	Import user_registration.sql; set demo passwords with the tool	1 admin, 2 students, $2y$ hashes	Pass
3	password_verify(set password, stored hash)	true; hash ≠ password	Pass
4	Open the three dashboards without logging in	Redirect to login, “Please log in to continue.”	Pass
5	Valid Student login	→ student_dashboard.php	Pass
6	Valid Admin login	→ admin_dashboard.php	Pass
7	Wrong password, unknown email, ' OR '1'='1, empty fields	“Invalid email or password.”	Pass
8	Login without CSRF token	Refused	Pass
9	Session cookie before vs. after login; reuse old ID	Different; old ID → login page	Pass
10	Student → admin_dashboard.php	403 Unauthorized Access	Pass
11	Admin → student_dashboard.php	403 Unauthorized Access	Pass
12	role=admin added to a student login	Ignored – student	Pass
13	GET logout.php / POST with wrong token	No logout	Pass
14	Logout (POST with token)	Logged-out message; cookie expired	Pass
15	Dashboards after logout; old cookie replayed	Login page	Pass
16	Timeout 2 s: requests within the limit; then 3 s idle	Kept alive; then timeout.php → login	Pass
17	Cache-Control on protected pages; Back button after logout (Edge)	no-store; login page shown	Pass
18	MySQL unreachable	General message; error in log only	Pass
19	db.php, includes/, tools/, tests/, config files	403	Pass
20	PHP error logs and Edge console	No passwords/hashes/warnings; only expected 403s	Pass

12. Problems Encountered and Fixes

- **Shared database and missing tables:** the original code used user_registration and queried students and events tables that do not exist there; the practical now has its own studenthub_prac10 and only uses the users table.
- **Session ID shown in the page:** the original dashboards printed session_id(). It was removed; regeneration is shown with fingerprints instead.

- **Messages from the address:** login.php printed ?error=… text; any text could be shown. Messages are now chosen from a fixed list.
- **Remember-me cookie:** it logged users in again automatically and so bypassed the inactivity timeout and logout; it was removed (not part of the requirements).
- **Time zone:** XAMPP's php.ini uses Europe/Berlin, so the dashboard showed a login time 4½ hours behind MySQL; db.php now sets Asia/Kolkata.
- **Screenshot automation:** the first capture reused a session from an interrupted run, so the login screenshots showed a dashboard; they were taken again with a fresh Edge profile.

13. Conclusion

The login system authenticates users with prepared statements and password_verify(), shows one generic message for every failed login, regenerates the session ID after login, and stores the user's role from the database in the server-side session. Role-based access control is enforced on the server for every protected page: a Student cannot open the Admin dashboard and an Admin cannot open the Student dashboard (HTTP 403 “Unauthorized Access”). Inactive sessions end through timeout.php, logout destroys the session completely, and no-store headers stop the Back button from showing a dashboard afterwards. All nine required scenarios were captured in Edge and all 76 automated checks passed. Remaining limitations: there is no limit on login attempts, and a role changed in the database takes effect at the next login.

14. References

- PHP Manual – password_verify: https://www.php.net/manual/en/function.password-verify.php
- PHP Manual – session_regenerate_id: https://www.php.net/manual/en/function.session-regenerate-id.php
- PHP Manual – Session security: https://www.php.net/manual/en/features.session.security.management.php
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- OWASP – Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP – Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder Practical-10)